Sistema de Moeda Estudantil

Laboratório de Desenvolvimento de Software — Sprint 3

O que foi feito e uma oficina rápida das tecnologias

Lucas Lima Magalhães Lafetá Chaves

Engenharia de Software — PUC Minas | Profa. Milena Menezes Adão

Como o sistema funciona



Professor

Recebe 1.000 moedas por semestre e reconhece alunos com moedas e uma mensagem.

envia moedas



Aluno

Acumula as moedas, consulta o extrato e troca por vantagens das empresas.

resgata vantagem



Empresa parceira

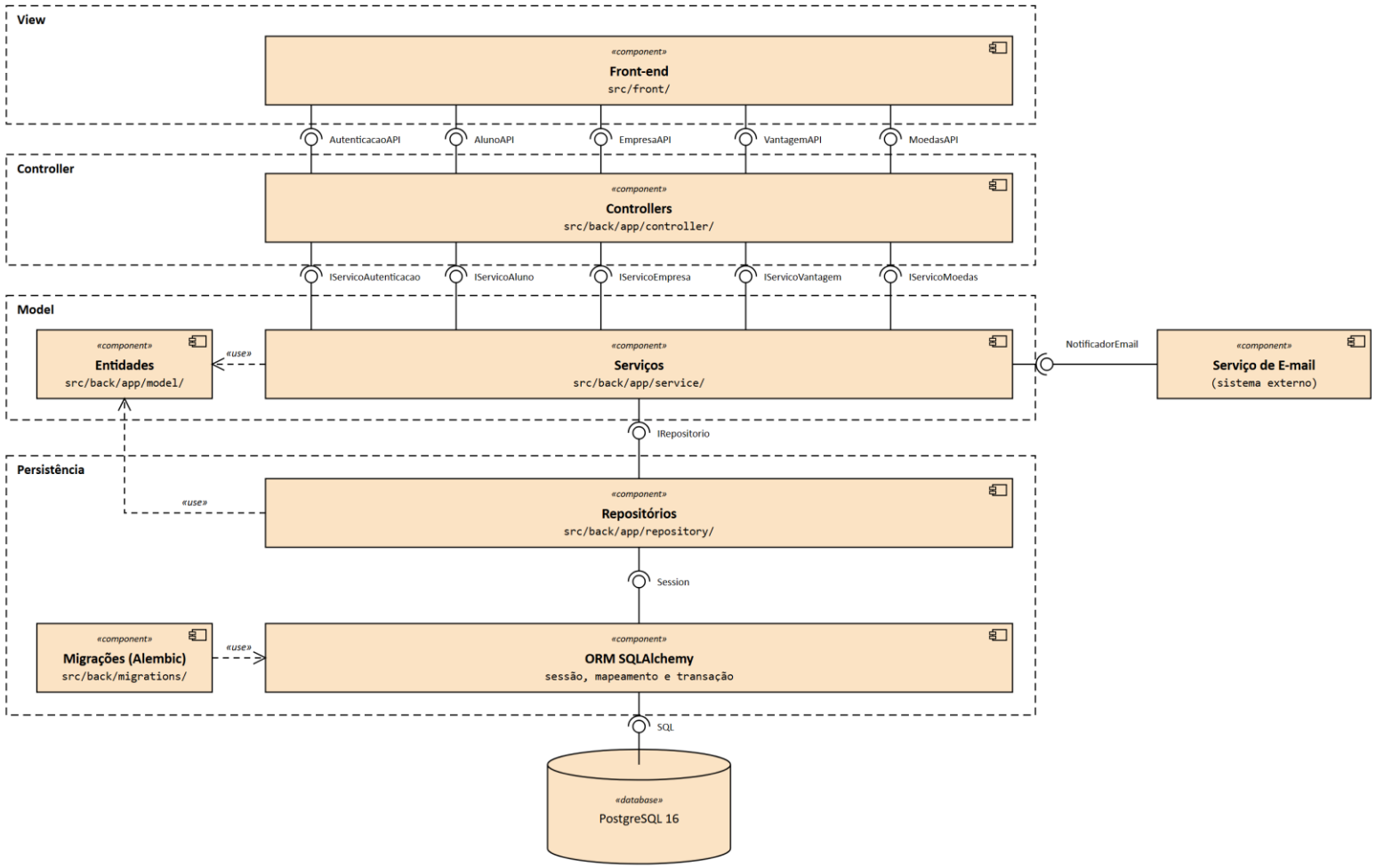
Cadastra vantagens com foto e custo em moedas e confere o cupom do aluno.



Todos os requisitos do enunciado

Login por perfil, envio de moedas com mensagem, extrato, vantagens com foto, resgate com código de cupom e e-mails ao aluno e à empresa.

Arquitetura em camadas (MVC)



View

React + TypeScript consome a API por HTTP

Controller

FastAPI: rotas, validação e perfis

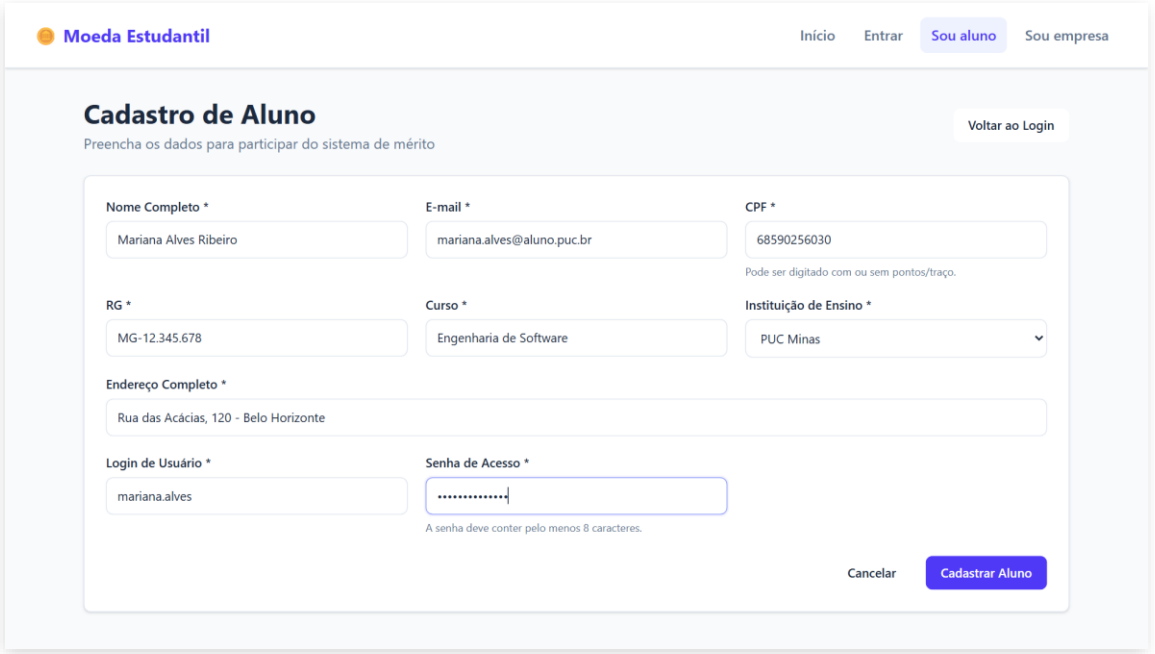
Model

Serviços de regra de negócio e entidades

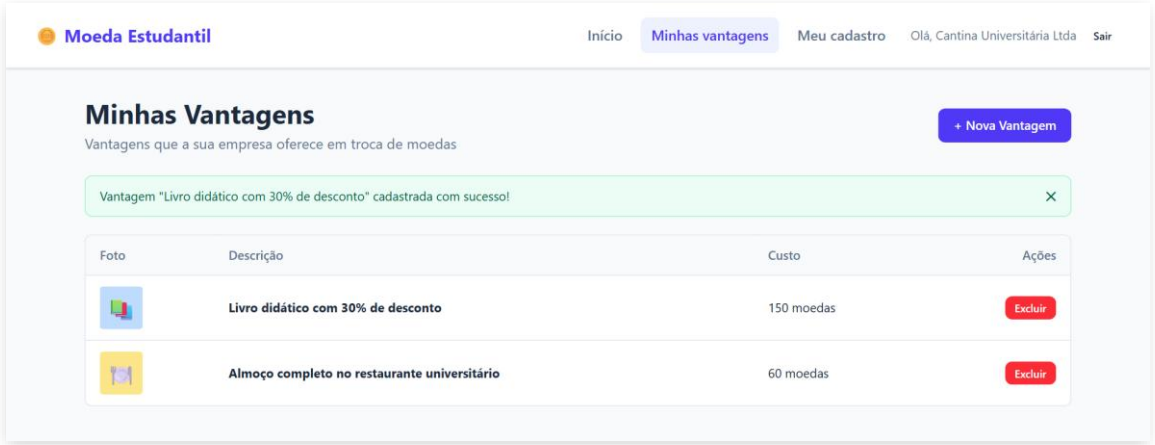
Persistência

Repositórios, SQLAlchemy e PostgreSQL

Demonstração: cadastro, login e vantagens



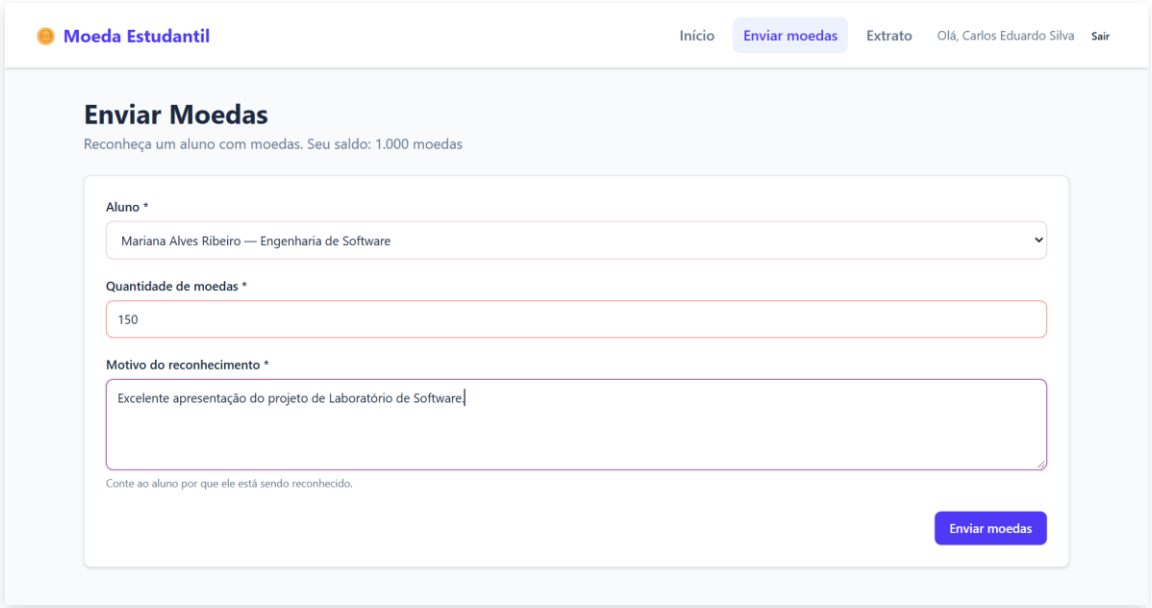
Cadastro do aluno, com erros por campo



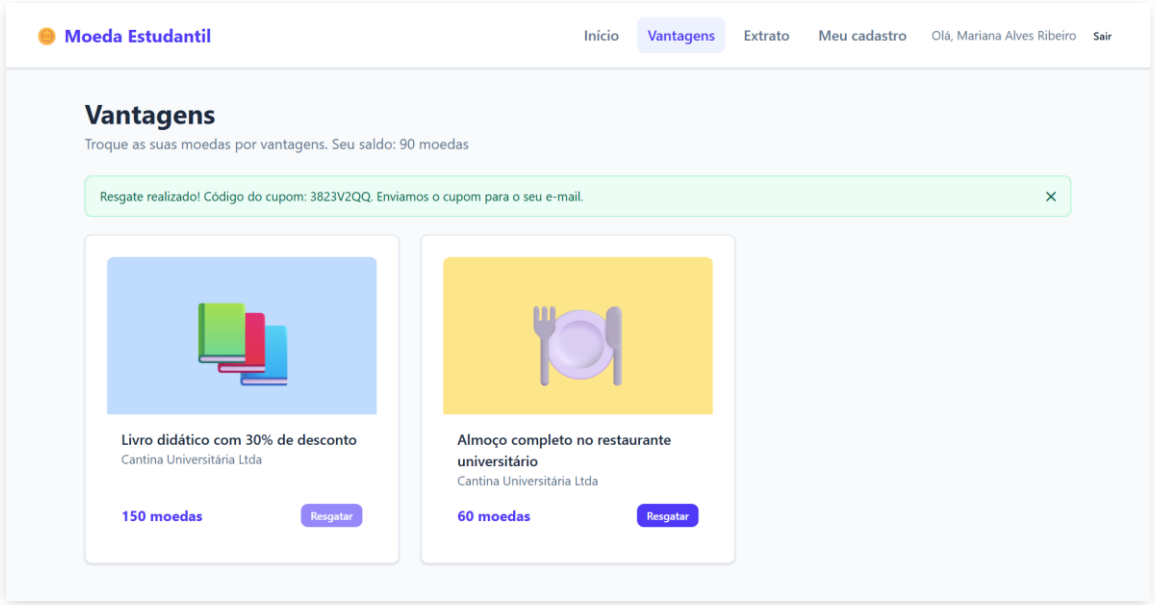
A empresa cadastra vantagens com foto

Cada perfil só enxerga as próprias telas: a empresa não acessa o extrato e o aluno não acessa o envio de moedas.

Demonstração: envio de moedas e resgate



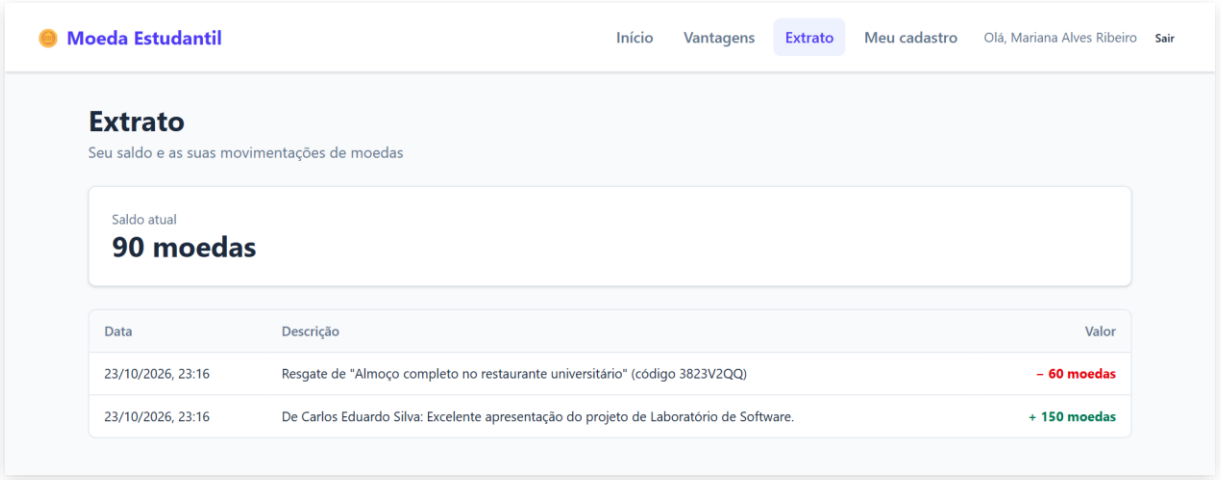
Professor: aluno, valor e motivo obrigatório



Aluno: resgate com o código do cupom

O saldo é conferido antes de cada envio e de cada resgate, e o botão de resgatar fica desabilitado sem saldo.

Extrato e e-mails



Extrato do aluno: o que entrou e o que saiu

Cupom do aluno

Código do cupom: 3823V2QQ

Conferência da empresa

Código para conferência: 3823V2QQ

Os dois e-mails levam o mesmo código. O envio usa o SMTP do Gmail configurado no .env; sem credenciais, o texto aparece no log da API.

Oficina 1: Docker Compose

O que é

Docker empacota cada serviço em um contêiner. O Compose descreve e liga vários contêineres em um arquivo.

Como usar

1. Descrever banco, API e front no docker-compose.yml
2. Subir tudo: docker compose up -d --build
3. Ver os registros: docker compose logs api
4. Limpar tudo: docker compose down -v

```
services:
  db:
    image: postgres:16-alpine
  api:
    build: ./src/back
    depends_on:
      db:
        condition: service_healthy
    command: >
      sh -c "alembic upgrade head
      && python -m app.tarefas.seed
      && uvicorn app.main:app --port 8000"
  front:
    build: ./src/front
    ports: ["5173:80"]
```

docker-compose.yml (trecho resumido e reformatado)

Oficina 2: FastAPI e Pydantic

O que é

FastAPI cria APIs REST em Python. Pydantic valida e converte os dados que entram e saem.

Como usar

1. Declarar a rota com o decorador
2. Receber um schema Pydantic no corpo
3. Injetar serviço e perfil com Depends
4. Abrir http://localhost:8000/docs

```
@router.post("/enviar", status_code=201,
             response_model=EnvioMoedasRespostaSchema)
def enviar_moedas(
    dados: EnvioMoedasCriarSchema,
    professor: Usuario = Depends(
        exigir_perfil(TipoUsuario.PROFESSOR)),
    servico: ServicoMoedas = Depends(
        obter_servico_moedas),
):
    envio = servico.enviar_moedas(
        professor.id_usuario, dados.model_dump())
    return EnvioMoedasRespostaSchema(...)
```

src/back/app/controller/rotas/moedas.py (trecho resumido e reformatado)

Oficina 3: SQLAlchemy, PostgreSQL e Alembic

O que é

O ORM SQLAlchemy mapeia classes em tabelas do PostgreSQL. O Alembic versiona o esquema em migrações.

Como usar

1. Declarar a entidade com mapped_column
2. Esconder o banco atrás de uma interface (Repository)
3. Criar as tabelas: alembic upgrade head
4. Travar o saldo: with_for_update()

```
class IRepositorioConta(ABC):
    @abstractmethod
    def buscar_por_usuario(
        self, id_usuario: int,
        para_atualizar: bool = False,
    ) -> Conta | None: ...

class RepositorioConta(IRepositorioConta):
    def buscar_por_usuario(
        self, id_usuario, para_atualizar=False):
        consulta = select(Conta).where(
            Conta.id_usuario == id_usuario)
        if para_atualizar:
            consulta = consulta.with_for_update()
        return self.sessao.execute(
            consulta).scalar_one_or_none()
```

src/back/app/repository (trecho resumido e reformatado)

Oficina 4: Login com JWT e bcrypt

O que é

bcrypt guarda a senha como hash com salt. O JWT é um token assinado que identifica o usuário a cada requisição.

Como usar

1. POST /autenticacao/entrar devolve o token
2. O front envia Authorization: Bearer <token>
3. Cada rota confere o perfil do usuário
4. Token inválido ou vencido: 401 e volta ao login

```
def emitir_token(self, usuario) -> str:
    agora = datetime.now(timezone.utc)
    carga = {
        "sub": str(usuario.id_usuario),
        "tipo": usuario.tipo_usuario.value,
        "iat": agora,
        "exp": agora + timedelta(
            minutes=self.validade_minutos),
    }
    return jwt.encode(
        carga, self.segredo, algorithm="HS256")

# senha: bcrypt.hashpw(senha, bcrypt.gensalt())
```

src/back/app/service/servico_autenticacao.py (trecho resumido e reformatado)

Oficina 5: React, Vite e Tailwind

O que é

React monta a interface com componentes e hooks. O Vite compila e serve. O Tailwind estiliza com classes direto no JSX.

Como usar

1. Cada tela é um componente; a lógica fica em hooks
2. npm run dev para ver, npm test para testar
3. Estilizar com classes: bg-indigo-600 text-white
4. O roteador protege cada tela por perfil

```
export const RotaProtegida = (
  { perfis, children }: Props
) => {
  const { usuario } = useAutenticacao();
  if (!usuario) {
    return <Navigate to="/entrar" replace />;
  }
  if (!perfis.includes(usuario.tipo_usuario)) {
    return <Navigate to="/" replace />;
  }
  return <>{children}</>;
};

primario: 'bg-indigo-600 text-white',
```

src/front/src/componentes/acesso/RotaProtegida.tsx e utils/estilos.ts (trecho resumido)

Qualidade e encerramento

207

testes do back-end

147

testes do front-end

46

verificações no navegador

- Envios e resgates simultâneos testados contra o PostgreSQL real; o saldo nunca fica negativo.
- Próximos passos: agendar o crédito do semestre e validar o envio real pelo Gmail.

```
docker compose up -d --build
# front: localhost:5173
# API:   localhost:8000/docs
```

github.com/lucaslmchaves/sistema-moeda-estudantil